MODUL PRAKTIKUM 2

DEKOMPOSISI DOMAIN, ISOLASI BASIS DATA TERDISTRIBUSI, DAN KOMUNIKASI SINKRON INTER-SERVICE (gRPC vs RESTful API)

1. INFORMASI & METADATA PRAKTIKUM

Mata Kuliah : Arsitektur Mikroservis (MK41)

Pertemuan RPS : Pertemuan 1, 2, dan 3 (Blok Fondasi Desain Terdistribusi & Komunikasi Sinkron)

Beban Kredit : 2 SKS Praktikum

Alokasi Waktu : 120 Menit

Model Pembelajaran : Simulasi Sistem Terpadu Berbasis Tim

2. INDIKATOR KETERCAPAIAN

1. **Indikator 1.1 (C4):** Ketepatan dekomposisi entitas SIMRS monolitik menjadi 4 *Bounded Context* mandiri tanpa adanya *Foreign Key* fisik antarlayanan.
2. **Indikator 1.2 (P3):** Keberhasilan konfigurasi dan inisialisasi 4 instance DBMS terisolasi (*multi-instance* PostgreSQL, MySQL, MongoDB) pada Docker virtual network terisolasi tanpa kegagalan *phantom mount*.
3. **Indikator 1.3 (C4):** Ketepatan membuktikan fenomena *Shared Database Anti-Pattern* (deadlock/locking contention) dan memvalidasi batas isolasi kredensial jaringan.
4. **Indikator 2.1 (C3):** Ketepatan penulisan kontrak interface .proto (IDL) dan kompilasi *stub/skeleton* lintas bahasa (Go dan TypeScript).
5. **Indikator 2.2 (P4):** Keberhasilan implementasi pemanggilan RPC sinkron *point-to-point* antara *Appointment Service* dan *Pharmacy Service* dengan penanganan status error terstandarisasi.
6. **Indikator 2.3 (C4):** Ketepatan analisis empiris komparasi performa (latensi p50/p95/p99, throughput RPS, dan ukuran byte payload) antara gRPC (HTTP/2 binary) dan RESTful API (HTTP/1.1 JSON) menggunakan k6.

3. DASAR TEORI OPERASIONAL

3.1. Strategic Domain-Driven Design (DDD) & Bounded Context

Dalam sistem monolitik rumah sakit, basis data biasanya dirancang dengan relasi foreign key global: tabel rekam_medis merujuk ke pasien, pasien merujuk ke asuransi, dan resep

merujuk ke stok_obat. Ketika volume transaksi meningkat, penguncian baris (*row-level lock*) pada satu tabel dapat melumpuhkan seluruh transaksi modul lain.

Domain-Driven Design (DDD) memisahkan sistem ke dalam **Bounded Context**, di mana setiap model data hanya berlaku di dalam batas konteksnya:

- **Patient Context:** Mengelola siklus hidup data demografi dan kredensial keamanan. Identitas pasien direpresentasikan secara lengkap (*full entity*).
- **Appointment Context:** Mengelola ketersediaan dokter dan antrean poliklinik. Entitas pasien di sini hanya direpresentasikan oleh sebuah **Logical Identifier** (patient_id bertipe UUID), bukan relasi foreign key fisik.
- **Pharmacy Context:** Mengelola persediaan batch obat dan pemenuhan resep. Obat dimodelkan sebagai dokumen agregat yang menyimpan riwayat nomor batch dan tanggal kedaluwarsa.
- **Billing Context:** Mengelola faktur tagihan dan verifikasi klaim asuransi. Tagihan merujuk secara logis ke patient_id dan appointment_id.

3.2. Pola Database-per-Service & Mitigasi Shared-Database Anti-Pattern

Pola **Database-per-Service** menjamin bahwa mikroservis memiliki skema penyimpanan mandiri yang tertutup rapat. Keuntungan utamanya:

1. **Otonomi Skema:** Perubahan struktur tabel pada layanan pasien tidak memerlukan koordinasi rilis atau migrasi skema pada basis data farmasi maupun tagihan.
2. **Polyglot Persistence:** Layanan farmasi dapat memakai MongoDB untuk menyimpan dokumen resep yang fleksibel, sementara layanan tagihan menggunakan MySQL dengan transaksi ACID ketat.
3. **Ketahanan Kegagalan (Fault Isolation):** Gangguan basis data pada layanan tagihan tidak menyebabkan layanan gawat darurat dan poliklinik terhenti.

Sebaliknya, **Shared Database Anti-Pattern** (mengakses basis data yang sama langsung via query SQL) menimbulkan bahaya:

- **Coupling Waktu Nyata (Runtime Coupling):** Transaksi panjang (*long-running transaction*) dari modul laporan keuangan mengunci baris data pasien, menyebabkan pendaftaran rawat jalan mengalami timeout.
- **Pelanggaran Batas Enkapsulasi:** Layanan lain dapat mengubah status transaksi secara langsung lewat SQL bypass tanpa melalui validasi aturan bisnis (*business invariants*).

3.3. Komunikasi Sinkron Inter-Service: HTTP/1.1 REST vs HTTP/2 gRPC

Untuk kebutuhan sinkronisasi data seketika (seperti verifikasi ketersediaan stok obat saat pembuatan janji temu klinik), mikroservis membutuhkan protokol komunikasi sinkron *point-to-point*.

4. KEBUTUHAN SISTEM & ENVIRONMENT

4.1. Spesifikasi Perangkat Lunak Laboratorium

- **Sistem Operasi:** Linux (Ubuntu 22.04 LTS direkomendasikan) / macOS 13+ / Windows 11 Pro dengan WSL2 aktif.
- **Container Platform:** Docker Engine v25.0+ dan Docker Compose v2.24+.
- **Compiler & Runtime:**
  - Go Programming Language v1.22+
  - Node.js v20.x LTS & npm v10.x
  - Protocol Buffer Compiler (protoc) v25+
- **Alat Pengujian & Benchmarking:**
  - k6 Load Testing Tool v0.49+
  - Postman v10+ (mendukung gRPC request) atau BloomRPC
  - DBeaver Universal Database Manager Community Edition
- **Plugin protoc yang Wajib Terpasang:**
  - protoc-gen-go (v1.33+)
  - protoc-gen-go-grpc (v1.3+)
  - ts-proto (v1.170+)

4.2. Struktur Direktori Proyek Tim (Monorepo MedCore)

Seluruh tim bekerja di dalam satu struktur direktori bersama sebagai berikut:

```
medcore-workspace/
├── .env
├── docker-compose.db.yml
├── proto/
│   ├── clinical.proto
│   └── pharmacy.proto
├── services/
│   ├── patient-service/      # Mahasiswa A (Node.js/TypeScript)
│   │   ├── package.json
│   │   ├── tsconfig.json
│   │   ├── src/
│   │   └── ddl/init.sql
│   ├── appointment-service/  # Mahasiswa B (Go)
│   │   ├── go.mod
│   │   ├── main.go
│   │   └── ddl/init.sql
│   ├── pharmacy-service/     # Mahasiswa C (Go)
│   │   ├── go.mod
│   │   ├── main.go
│   │   └── ddl/init.js
│   └── billing-service/      # Mahasiswa D (Go)
│       ├── go.mod
│       └── ddl/init.sql
└── tests/
    └── benchmark/
```

```
├── k6-grpc-test.js
└── k6-rest-test.js
```

5. LANGKAH PRAKTIKUM TERPANDU

SESI I: DEKOMPOSISI DOMAIN & ISOLASI MULTI-INSTANCE DBMS

Timeline Sesi I (Strict Timeboxing):

- Inisialisasi Workspace Tim, Git Monorepo, & Pembuatan Stub/Placeholder.
- Penulisan DDL Mandiri per Bounded Context (Mahasiswa A, B, C, D).
- Checkpoint Git Sync & Eksekusi Orkestrasi Multi-Instance DBMS.
- Pengujian Isolasi Kredensial (L7) & Simulasi Table Lock Contention.
- Studi Kasus Mandiri Sesi I: Protokol Gawat Darurat "Break-The-Glass".
- Logbook Verification & Dosen/Asisten Lab Sign-Off.

Langkah 1.1: Setup Workspace & Anti-Phantom Mount Stubs

Penanggung Jawab: Mahasiswa D (DevOps Lead) dibantu seluruh anggota.

Untuk mencegah Docker membuat folder palsu (*phantom directory*) saat file DDL belum ditulis, seluruh placeholder file dibuat terlebih dahulu menggunakan perintah touch.

1. Jalankan di root terminal workspace:

```
mkdir -p medcore-workspace/{proto,tests/benchmark}
mkdir -p medcore-workspace/services/patient-service/ddl
mkdir -p medcore-workspace/services/appointment-service/ddl
mkdir -p medcore-workspace/services/pharmacy-service/ddl
mkdir -p medcore-workspace/services/billing-service/ddl

# WAJIB: Buat berkas stub kosong agar mount Docker tidak membuat
direktori kosong!
touch medcore-workspace/services/patient-service/ddl/init.sql
touch medcore-workspace/services/appointment-service/ddl/init.sql
touch medcore-workspace/services/pharmacy-service/ddl/init.js
touch medcore-workspace/services/billing-service/ddl/init.sql
touch medcore-workspace/.env

cd medcore-workspace
git init
git checkout -b main
```

2. Konfigurasi kredensial bersama pada berkas .env:

```
# File: medcore-workspace/.env
# PORT MAPPING MESIN HOST (Tidak bentrok karena port host dibedakan)
```

```
HOST_PORT_PATIENT_DB=5432
HOST_PORT_APPOINTMENT_DB=5433
HOST_PORT_PHARMACY_DB=27017
HOST_PORT_BILLING_DB=3306

# KREDENSIAL PATIENT DB (PostgreSQL Engine 1)
PATIENT_DB_USER=usr_patient_svc
PATIENT_DB_PASS=SecuredPassPatient2026!
PATIENT_DB_NAME=medcore_patient_db

# KREDENSIAL APPOINTMENT DB (PostgreSQL Engine 2)
APPOINTMENT_DB_USER=usr_appointment_svc
APPOINTMENT_DB_PASS=SecuredPassAppt2026!
APPOINTMENT_DB_NAME=medcore_appointment_db

# KREDENSIAL PHARMACY DB (MongoDB Engine)
MONGO_ROOT_USER=adm_pharmacy_svc
MONGO_ROOT_PASS=SecuredPassPharm2026!
PHARMACY_DB_NAME=medcore_pharmacy_db

# KREDENSIAL BILLING DB (MySQL Engine)
BILLING_DB_USER=usr_billing_svc
BILLING_DB_PASS=SecuredPassBilling2026!
BILLING_DB_NAME=medcore_billing_db
```

3. Buat berkas orkestrasi docker-compose.db.yml:

```
networks:
  medcore-net:
    driver: bridge
    ipam:
      config:
        - subnet: 172.28.0.0/16

services:
  patient-db:
    image: postgres:16-alpine
    container_name: medcore-patient-db
    environment:
      POSTGRES_USER: ${PATIENT_DB_USER}
      POSTGRES_PASSWORD: ${PATIENT_DB_PASS}
      POSTGRES_DB: ${PATIENT_DB_NAME}
    ports:
      - "${HOST_PORT_PATIENT_DB}:5432"
```

```
    volumes:
      -
./services/patient-service/ddl/init.sql:/docker-entrypoint-initdb.d/init
.sql:ro
      - patient_db_vol:/var/lib/postgresql/data
    networks:
      medcore-net:
        ipv4_address: 172.28.0.11

  appointment-db:
    image: postgres:16-alpine
    container_name: medcore-appointment-db
    environment:
      POSTGRES_USER: ${APPOINTMENT_DB_USER}
      POSTGRES_PASSWORD: ${APPOINTMENT_DB_PASS}
      POSTGRES_DB: ${APPOINTMENT_DB_NAME}
    ports:
      - "${HOST_PORT_APPOINTMENT_DB}:5432"
    volumes:
      -
./services/appointment-service/ddl/init.sql:/docker-entrypoint-initdb.d/
init.sql:ro
      - appointment_db_vol:/var/lib/postgresql/data
    networks:
      medcore-net:
        ipv4_address: 172.28.0.12

  pharmacy-db:
    image: mongo:7.0-jammy
    container_name: medcore-pharmacy-db
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_ROOT_USER}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_ROOT_PASS}
      MONGO_INITDB_DATABASE: ${PHARMACY_DB_NAME}
    ports:
      - "${HOST_PORT_PHARMACY_DB}:27017"
    volumes:
      -
./services/pharmacy-service/ddl/init.js:/docker-entrypoint-initdb.d/init
.js:ro
      - pharmacy_db_vol:/data/db
    networks:
      medcore-net:
        ipv4_address: 172.28.0.13
```

```
  billing-db:
    image: mysql:8.0-debian
    container_name: medcore-billing-db
    command: --default-authentication-plugin=mysql_native_password
    environment:
      MYSQL_ROOT_PASSWORD: RootMasterSecurity2026!
      MYSQL_DATABASE: ${BILLING_DB_NAME}
      MYSQL_USER: ${BILLING_DB_USER}
      MYSQL_PASSWORD: ${BILLING_DB_PASS}
    ports:
      - "${HOST_PORT_BILLING_DB}:3306"
    volumes:
      -
./services/billing-service/ddl/init.sql:/docker-entrypoint-initdb.d/init
.sql:ro
      - billing_db_vol:/var/lib/mysql
    networks:
      medcore-net:
        ipv4_address: 172.28.0.14

volumes:
  patient_db_vol:
  appointment_db_vol:
  pharmacy_db_vol:
  billing_db_vol:
```

Langkah 1.2: Penulisan DDL Mandiri per Bounded Context

A. Tugas Mahasiswa A: Patient & Identity DDL

Buka berkas services/patient-service/ddl/init.sql dan isi dengan skema mandiri:

```
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

CREATE TABLE patients (
    patient_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    national_id VARCHAR(16) UNIQUE NOT NULL,      -- NIK 16 Digit
    medical_record_no VARCHAR(15) UNIQUE NOT NULL, -- Nomor Rekam Medis
(No RM)
    full_name VARCHAR(150) NOT NULL,
    date_of_birth DATE NOT NULL,
    gender VARCHAR(10) CHECK (gender IN ('MALE', 'FEMALE')),
    blood_type VARCHAR(3),
```

```
    phone_number VARCHAR(20) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE user_credentials (
    credential_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    patient_id UUID REFERENCES patients(patient_id) ON DELETE RESTRICT,
    email VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    system_role VARCHAR(20) NOT NULL CHECK (system_role IN ('PATIENT',
'DOCTOR', 'NURSE', 'PHARMACIST')),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Seed Data Awal
INSERT INTO patients (patient_id, national_id, medical_record_no,
full_name, date_of_birth, gender, blood_type, phone_number)
VALUES
('a0000000-0000-0000-0000-000000000001', '5171010101850001',
'RM-2026-0001', 'I Wayan Sudarma', '1985-05-15', 'MALE', 'O',
'+6281234567890'),
('a0000000-0000-0000-0000-000000000002', '5171010202900002',
'RM-2026-0002', 'Ni Made Rai', '1990-11-20', 'FEMALE', 'A',
'+6281987654321');
```

B. Tugas Mahasiswa B: Appointment & Consultation DDL

Buka berkas services/appointment-service/ddl/init.sql. Perhatikan bahwa patient_id **bukan foreign key fisik**, melainkan referensi logis.

```
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

CREATE TABLE doctors (
    doctor_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    license_number VARCHAR(50) UNIQUE NOT NULL,
    doctor_name VARCHAR(150) NOT NULL,
    specialization VARCHAR(50) NOT NULL,
    is_active BOOLEAN DEFAULT TRUE
);

CREATE TABLE appointments (
    appointment_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    patient_id UUID NOT NULL, -- REFERENSI LOGIS KE PATIENT SERVICE
```

```
(DECOUPLED)
    doctor_id UUID REFERENCES doctors(doctor_id),
    schedule_time TIMESTAMP WITH TIME ZONE NOT NULL,
    queue_number INT NOT NULL,
    consultation_status VARCHAR(20) DEFAULT 'SCHEDULED'
        CHECK (consultation_status IN ('SCHEDULED', 'IN_PROGRESS',
'COMPLETED', 'CANCELLED')),
    clinical_notes TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Seed Data Awal
INSERT INTO doctors (doctor_id, license_number, doctor_name,
specialization)
VALUES
('b0000000-0000-0000-0000-000000000001', 'SIP-BALI-9901', 'dr. Ketut
Gede, Sp.PD', 'Penyakit Dalam'),
('b0000000-0000-0000-0000-000000000002', 'SIP-BALI-9902', 'dr. Putu Ayu,
Sp.A', 'Spesialis Anak');

INSERT INTO appointments (appointment_id, patient_id, doctor_id,
schedule_time, queue_number, consultation_status)
VALUES
('b1111111-1111-1111-1111-111111111111',
'a0000000-0000-0000-0000-000000000001',
'b0000000-0000-0000-0000-000000000001', CURRENT_TIMESTAMP + INTERVAL '1
hour', 1, 'SCHEDULED');
```

C. Tugas Mahasiswa C: Pharmacy & Medical Inventory Init Script

Buka berkas services/pharmacy-service/ddl/init.js:

```
db = db.getSiblingDB('medcore_pharmacy_db');

db.createCollection('drugs');
db.createCollection('prescriptions');

db.drugs.createIndex({ "drug_code": 1 }, { unique: true });
db.prescriptions.createIndex({ "appointment_id": 1 });

db.drugs.insertMany([
  {
    drug_code: "MED-AMX-500",
```

```
    name: "Amoxicillin 500mg",
    category: "Antibiotik",
    unit_price: 3500.0,
    total_stock: 500,
    batches: [
      { batch_no: "BATCH-2026A", stock: 200, exp_date: new
Date("2028-01-01") },
      { batch_no: "BATCH-2026B", stock: 300, exp_date: new
Date("2028-06-01") }
    ]
  },
  {
    drug_code: "MED-PCT-500",
    name: "Paracetamol 500mg",
    category: "Analgesik",
    unit_price: 1500.0,
    total_stock: 1000,
    batches: [
      { batch_no: "BATCH-PCT-01", stock: 1000, exp_date: new
Date("2029-01-01") }
    ]
  }
]);

db.prescriptions.insertOne({
  prescription_id: "RX-2026-0001",
  appointment_id: "b1111111-1111-1111-1111-111111111111", // Logical ID
dari Appointment DB
  patient_id: "a0000000-0000-0000-0000-000000000001",    // Logical ID
dari Patient DB
  items: [
    { drug_code: "MED-AMX-500", quantity: 15, instructions: "3x1 sehari
sesudah makan" },
    { drug_code: "MED-PCT-500", quantity: 10, instructions: "3x1 bila
demam" }
  ],
  status: "ISSUED",
  created_at: new Date()
});
```

D. Tugas Mahasiswa D: Billing & Claims DDL

Buka berkas services/billing-service/ddl/init.sql:

```
CREATE TABLE invoices (
```

```
    invoice_id VARCHAR(36) PRIMARY KEY,
    patient_id VARCHAR(36) NOT NULL,        -- Logical ID
    appointment_id VARCHAR(36) NOT NULL,    -- Logical ID
    total_amount DECIMAL(12,2) NOT NULL,
    payment_status VARCHAR(20) DEFAULT 'UNPAID',
    issued_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_appt (appointment_id),
    INDEX idx_patient (patient_id)
) ENGINE=InnoDB;

CREATE TABLE invoice_items (
    item_id VARCHAR(36) PRIMARY KEY,
    invoice_id VARCHAR(36) NOT NULL,
    description VARCHAR(255) NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    FOREIGN KEY (invoice_id) REFERENCES invoices(invoice_id) ON DELETE
CASCADE
) ENGINE=InnoDB;

INSERT INTO invoices (invoice_id, patient_id, appointment_id,
total_amount, payment_status)
VALUES ('inv-0001', 'a0000000-0000-0000-0000-000000000001',
'b1111111-1111-1111-1111-111111111111', 150000.00, 'UNPAID');

INSERT INTO invoice_items (item_id, invoice_id, description, amount)
VALUES
('item-1', 'inv-0001', 'Jasa Konsultasi Spesialis Penyakit Dalam',
100000.00),
('item-2', 'inv-0001', 'Biaya Administrasi Rawat Jalan', 50000.00);
```

Langkah 1.3: Checkpoint Sinkronisasi Git & Provisioning (Menit 00:50 – 01:05)

1. Seluruh anggota melakukan commit berkas DDL mereka ke cabang Git masing-masing dan melakukan *merge* ke cabang main.
2. Mahasiswa D memverifikasi integritas berkas di workstation host dan menyalakan seluruh cluster DBMS:

```
docker compose -f docker-compose.db.yml up -d
```

3. Verifikasi status operasional 4 basis data:

```
docker compose -f docker-compose.db.yml ps
```

Pastikan seluruh 4 kontainer berstatus Up.

Langkah 1.4: Pengujian Isolasi Kredensial & Simulasi Anti-Pattern (Menit 01:05 – 01:25)

A. Uji Isolasi Kredensial Database (Layer 7 Verification)

Mahasiswa A mencoba masuk ke database *Appointment DB* (milik Mahasiswa B) menggunakan user dari *Patient DB*:

```
docker exec -it medcore-patient-db psql -h 172.28.0.12 -U
usr_patient_svc -d medcore_appointment_db
```

Hasil yang Diharapkan: Sistem menolak koneksi dengan pesan:

FATAL: password authentication failed for user "usr_patient_svc"

Makna Arsitektural: Lapisan jaringan L3/L4 terhubung sempurna, namun autentikasi data terisolasi secara otonom (*Principle of Least Privilege*).

B. Simulasi Shared Database Anti-Pattern (Table Lock Contention)

Tunjukkan mengapa basis data tidak boleh dipakai bersama lintas fungsional:

1. Buka **Terminal 1** (Simulasi proses update rekam medis oleh Poliklinik):

```
docker exec -it medcore-appointment-db psql -U usr_appointment_svc -d
medcore_appointment_db
```

Jalankan transaksi yang sengaja digantung (*uncommitted*):

```
BEGIN;
SELECT * FROM appointments WHERE appointment_id =
'b1111111-1111-1111-1111-111111111111' FOR UPDATE;
-- JANGAN EKSEKUSI COMMIT DAHULU!
```

2. Buka **Terminal 2** (Simulasi proses pembatalan janji temu oleh Sistem Kasir/Billing yang mengakses DB yang sama):

```
docker exec -it medcore-appointment-db psql -U usr_appointment_svc -d
medcore_appointment_db
```

Jalankan perintah update:

```
UPDATE appointments SET consultation_status = 'CANCELLED' WHERE
appointment_id = 'b1111111-1111-1111-1111-111111111111';
```

Observasi: Terminal 2 akan membeku (*hanging*) menunggu lock dilepaskan. Inilah bukti empiris bahaya *Shared Database Anti-Pattern*.

3. Lepaskan lock pada **Terminal 1**:

```
ROLLBACK;
```

Terminal 2 seketika menyelesaikan update.

Langkah 1.5: Studi Kasus Mandiri Sesi I: Protokol "Break-The-Glass"

Problem Statement:

Seorang korban darurat trauma fisik masuk ke IGD tanpa identitas (Mr. X / Anonim).

1. Dokter IGD harus segera membuat pendaftaran janji temu (*Appointment*) dan mengeluarkan resep darurat (*Pharmacy*) tanpa menunggu proses registrasi KTP/NIK selesai.
2. Tim ditantang: Bagaimana skema *Logical ID* Anda memfasilitasi kebutuhan ini tanpa melanggar konstrain database?
3. **Instruksi Eksekusi Tim:**
   - **Mahasiswa B:** Masukkan data appointment baru menggunakan patient_id sementara bertipe acak: 00000000-9999-9999-9999-000000000000 dengan status IN_PROGRESS dan catatan 'EMERGENCY UNIDENTIFIED PATIENT'.
   - **Mahasiswa C:** Terbitkan resep di MongoDB merujuk ke ID sementara tersebut.
   - **Mahasiswa A:** Setelah 10 menit, data riil pasien terdaftar (NIK: 5171099999990001). Catat data pasien baru ini di *Patient DB*.
   - **Diskusi Tim:** Buktikan mengapa arsitektur decoupled ini tidak gagal (*no foreign key error*), dan rumuskan query rekonsiliasi yang harus dikirimkan ke layanan lain!

Langkah 1.6: Logbook Verification Sesi I

Tunjukkan kepada Dosen/Asisten:

1. Keempat database menyala di Docker.
2. Hasil eksekusi penolakan koneksi kredensial lintas database.
3. Rekaman terminal lock contention.
4. Tanda tangani Logbook Bagian I.

SESI II: INTERFACE CONTRACTS, CODE GENERATION, & IMPLEMENTASI gRPC SINKRON

Timeline Sesi II (Strict Timeboxing):

- Penyusunan Kontrak .proto Bersama (pharmacy.proto & clinical.proto).
- Kompilasi Otomatis Stub/Skeleton Lintas Bahasa (Go & TypeScript).
- Implementasi gRPC Server (Mahasiswa C) & gRPC Client (Mahasiswa B).
- Live End-to-End Inter-Service Synchronous Verification.
- Studi Kasus Mandiri Sesi II: gRPC Standard Status Code & Error Handling.
- Logbook Verification & Dosen/Asisten Lab Sign-Off.

Langkah 2.1: Penyusunan Kontrak Antarmuka Protocol Buffers

Target: Mahasiswa B dan C menyepakati kontrak pengecekan stok dan validasi resep.

Buat berkas kontrak antarmuka di proto/pharmacy.proto:

```
syntax = "proto3";

package medcore.pharmacy.v1;

option go_package = "services/pharmacy-service/pb;pharmacypb";

service PharmacyService {
  // RPC Unary untuk verifikasi ketersediaan stok obat oleh layanan
klinik
  rpc CheckDrugAvailability (CheckDrugRequest) returns
(CheckDrugResponse);

  // RPC Unary untuk mengunci stok sementara saat resep dibuat
  rpc ReservePrescriptionStock (ReserveStockRequest) returns
(ReserveStockResponse);
}

message CheckDrugRequest {
  string drug_code = 1;
  int32 quantity_needed = 2;
}

message CheckDrugResponse {
  string drug_code = 1;
  bool is_available = 2;
  int32 current_stock = 3;
  double unit_price = 4;
  string message = 5;
}

message StockItem {
  string drug_code = 1;
  int32 quantity = 2;
}

message ReserveStockRequest {
  string prescription_id = 1;
  string appointment_id = 2;
  repeated StockItem items = 3;
}
```

```
message ReserveStockResponse {
  bool reservation_success = 1;
  string transaction_reference = 2;
  string error_detail = 3;
}
```

Langkah 2.2: Kompilasi Stub/Skeleton Menggunakan protoc

A. Kompilasi untuk Layanan Go (Mahasiswa B & C)

1. Di root workspace, inisialisasi module untuk layanan farmasi dan appointment:

```
# Inisialisasi Pharmacy Service
cd medcore-workspace/services/pharmacy-service
go mod init pharmacy-service
go get google.golang.org/grpc@v1.62.1
go get google.golang.org/protobuf@v1.33.0
go get go.mongodb.org/mongo-driver/mongo@v1.14.0

# Inisialisasi Appointment Service
cd ../appointment-service
go mod init appointment-service
go get google.golang.org/grpc@v1.62.1
go get google.golang.org/protobuf@v1.33.0
go get github.com/lib/pq@v1.10.9
```

2. Kembali ke root medcore-workspace, buat folder pb dan jalankan kompilasi:

```
mkdir -p services/pharmacy-service/pb
mkdir -p services/appointment-service/pb

protoc --proto_path=proto \
  --go_out=services/pharmacy-service/pb --go_opt=paths=source_relative \
  --go-grpc_out=services/pharmacy-service/pb
--go-grpc_opt=paths=source_relative \
  proto/pharmacy.proto

# Salin kode pb yang sama ke Appointment Service agar sinkron
cp -r services/pharmacy-service/pb/* services/appointment-service/pb/
```

Verifikasi: Pastikan terbentuk berkas pharmacy.pb.go dan pharmacy_grpc.pb.go di kedua direktori layanan.

Langkah 2.3: Implementasi gRPC Server pada Pharmacy Service

Penanggung Jawab: Mahasiswa C (Didampingi Mahasiswa D).

Buka berkas services/pharmacy-service/main.go dan tulis implementasi server RPC yang memeriksa stok ke MongoDB:

```
package main

import (
	"context"
	"fmt"
	"log"
	"net"
	"time"

	pb "pharmacy-service/pb"

	"go.mongodb.org/mongo-driver/bson"
	"go.mongodb.org/mongo-driver/mongo"
	"go.mongodb.org/mongo-driver/mongo/options"
	"google.golang.org/grpc"
	"google.golang.org/grpc/codes"
	"google.golang.org/grpc/status"
)

type server struct {
	pb.UnimplementedPharmacyServiceServer
	mongoClient *mongo.Client
}

type DrugDoc struct {
	DrugCode   string  `bson:"drug_code"`
	Name       string  `bson:"name"`
	UnitPrice  float64 `bson:"unit_price"`
	TotalStock int32   `bson:"total_stock"`
}

func (s *server) CheckDrugAvailability(ctx context.Context, req
*pb.CheckDrugRequest) (*pb.CheckDrugResponse, error) {
	log.Printf("[gRPC Server] Memeriksa stok obat: %s, jumlah diminta:
%d", req.GetDrugCode(), req.GetQuantityNeeded())

	coll :=
s.mongoClient.Database("medcore_pharmacy_db").Collection("drugs")

	var drug DrugDoc
```

```
	err := coll.FindOne(ctx, bson.M{"drug_code": req.GetDrugCode()}).Decode(&drug)
	if err != nil {
		if err == mongo.ErrNoDocuments {
			return nil, status.Errorf(codes.NotFound, "Obat dengan kode %s tidak ditemukan dalam inventaris", req.GetDrugCode())
		}
		return nil, status.Errorf(codes.Internal, "Database error: %v", err)
	}

	if drug.TotalStock < req.GetQuantityNeeded() {
		return &pb.CheckDrugResponse{
			DrugCode:     drug.DrugCode,
			IsAvailable:  false,
			CurrentStock: drug.TotalStock,
			UnitPrice:    drug.UnitPrice,
			Message:      fmt.Sprintf("Stok tidak mencukupi. Tersedia: %d, Dibutuhkan: %d", drug.TotalStock, req.GetQuantityNeeded()),
		}, nil
	}

	return &pb.CheckDrugResponse{
		DrugCode:     drug.DrugCode,
		IsAvailable:  true,
		CurrentStock: drug.TotalStock,
		UnitPrice:    drug.UnitPrice,
		Message:      "Stok obat mencukupi",
	}, nil
}

func main() {
	// Koneksi ke MongoDB kontainer
	ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
	defer cancel()

	mongoURI := "mongodb://adm_pharmacy_svc:SecuredPassPharm2026!@localhost:27017/medcore_pharmacy_db?authSource=admin"
	client, err := mongo.Connect(ctx, options.Client().ApplyURI(mongoURI))
	if err != nil {
```

```
            log.Fatalf("Gagal terhubung ke MongoDB: %v", err)
      }

      lis, err := net.Listen("tcp", ":50051")
      if err != nil {
            log.Fatalf("Gagal membuka port 50051: %v", err)
      }

      s := grpc.NewServer()
      pb.RegisterPharmacyServiceServer(s, &server{mongoClient: client})

      log.Println("==================================================")
      log.Println("MedCore Pharmacy gRPC Service aktif pada port
:50051")
      log.Println("==================================================")
      if err := s.Serve(lis); err != nil {
            log.Fatalf("Gagal menjalankan gRPC server: %v", err)
      }
}
```

Jalankan gRPC Server:

```
cd medcore-workspace/services/pharmacy-service
go run main.go
```

Langkah 2.4: Implementasi gRPC Client pada Appointment Service (Menit 01:10 – 01:30)

Penanggung Jawab: Mahasiswa B.

Buka berkas services/appointment-service/main.go. Layanan ini memanggil gRPC Server *Pharmacy Service* secara sinkron sebelum mengonfirmasi status antrean:

```
package main

import (
      "context"
      "log"
      "time"

      pb "appointment-service/pb"

      "google.golang.org/grpc"
      "google.golang.org/grpc/credentials/insecure"
      "google.golang.org/grpc/status"
```

```
)

func main() {
	log.Println("[Appointment Service] Menginisialisasi koneksi gRPC
ke Pharmacy Service...")

	// Koneksi gRPC Channel (HTTP/2 Multiplexed)
	conn, err := grpc.Dial("localhost:50051",
grpc.WithTransportCredentials(insecure.NewCredentials()))
	if err != nil {
		log.Fatalf("Tidak dapat membentuk koneksi ke Pharmacy
Service: %v", err)
	}
	defer conn.Close()

	client := pb.NewPharmacyServiceClient(conn)

	// Skenario 1: Meminta obat yang tersedia (Amoxicillin 500mg,
butuh 20)
	requestValid := &pb.CheckDrugRequest{
		DrugCode:       "MED-AMX-500",
		QuantityNeeded: 20,
	}

	ctx, cancel := context.WithTimeout(context.Background(),
2*time.Second)
	defer cancel()

	log.Printf("[RPC Call] Mengirim permintaan validasi obat: %s",
requestValid.DrugCode)
	resp, err := client.CheckDrugAvailability(ctx, requestValid)
	if err != nil {
		st, _ := status.FromError(err)
		log.Fatalf("RPC Gagal: Code=%s, Message=%s", st.Code(),
st.Message())
	}

	log.Println("================== HASIL RESPON gRPC
==================")
	log.Printf("Kode Obat      : %s", resp.GetDrugCode())
	log.Printf("Tersedia       : %t", resp.GetIsAvailable())
	log.Printf("Stok Aktual    : %d", resp.GetCurrentStock())
	log.Printf("Harga Satuan   : Rp %.2f", resp.GetUnitPrice())
	log.Printf("Catatan Sistem : %s", resp.GetMessage())
```

```
log.Println("============================================================")
}
```

Jalankan Client pada terminal kedua:

```
cd medcore-workspace/services/appointment-service
go run main.go
```

Hasil: Layanan Appointment menerima informasi ketersediaan stok obat secara biner dengan latensi sub-milidetik.

Langkah 2.5: Studi Kasus Mandiri Sesi II: gRPC Error Standardization (Menit 01:30 – 01:45)

Problem Statement:

Saat dokter di poliklinik salah menginput kode obat (misal: MED-TIDAK-ADA), atau meminta obat sebanyak 10.000 butir melebihi stok yang ada, client tidak boleh menerima error generic 500 Internal Server Error. Sistem harus membedakan error not found (codes.NotFound) dan resource exhausted (codes.ResourceExhausted).

Tugas Mandiri Mahasiswa B dan C:

1. Modifikasi services/pharmacy-service/main.go sehingga bila QuantityNeeded > 5000, server melemparkan status error status.Errorf(codes.ResourceExhausted, "Permintaan melebihi batas kuota farmasi").
2. Modifikasi services/appointment-service/main.go untuk menangkap kode error spesifik menggunakan status.FromError(err) dan mencetak log peringatan klinis yang berbeda sesuai kode status tersebut.

Langkah 2.6: Logbook Verification Sesi II

Tunjukkan kepada Dosen/Asisten:

1. File .proto yang telah disepakati.
2. Hasil eksekusi gRPC Unary RPC call antara Terminal Pharmacy dan Terminal Appointment.
3. Penanganan error NotFound dan ResourceExhausted.
4. Tanda tangani Logbook Bagian II.

SESI III: RESTful API BASELINE, BENCHMARKING, & NETWORK PAYLOAD PROFILING

Timeline Sesi III (Strict Timeboxing):

- Konstruksi Endpoint Baseline REST (HTTP/1.1 JSON) pada Pharmacy Service.

- Penulisan Skenario k6 & Eksekusi Uji Beban Terukur (50, 200, 500 VUs).
- Inspeksi Wire-Size Frame Payload Biner Protobuf vs String JSON.
- Studi Kasus Mandiri Sesi III: Simulasi Network Latency Injection & RPC Deadline Timeout.
- Rekapitulasi Data Komparasi, Git Freeze, & Penilaian Akhir Modul Terpadu.

Langkah 3.1: Konstruksi Endpoint Baseline RESTful API

Target: Menambahkan HTTP Server berbasis JSON pada Pharmacy Service dengan logika query yang identik sebagai instrumen komparasi.

Tambahkan handler REST pada services/pharmacy-service/main.go:

```
// Tambahkan import "encoding/json" dan "net/http" pada file main.go

func startRESTServer(mongoClient *mongo.Client) {
	http.HandleFunc("/api/v1/drugs/check", func(w http.ResponseWriter,
r *http.Request) {
		if r.Method != http.MethodGet {
			http.Error(w, "Method not allowed",
http.StatusMethodNotAllowed)
			return
		}

		drugCode := r.URL.Query().Get("drug_code")
		coll :=
mongoClient.Database("medcore_pharmacy_db").Collection("drugs")

		var drug DrugDoc
		err := coll.FindOne(r.Context(), bson.M{"drug_code":
drugCode}).Decode(&drug)
		if err != nil {
			w.Header().Set("Content-Type", "application/json")
			w.WriteHeader(http.StatusNotFound)
			json.NewEncoder(w).Encode(map[string]string{"error":
"Obat tidak ditemukan"})
			return
		}

		w.Header().Set("Content-Type", "application/json")
		json.NewEncoder(w).Encode(map[string]interface{}{
			"drug_code":     drug.DrugCode,
			"is_available":  drug.TotalStock >= 10,
			"current_stock": drug.TotalStock,
			"unit_price":    drug.UnitPrice,
			"message":       "Stok obat mencukupi",
```

```
            })
        })

        log.Println("MedCore Pharmacy REST Baseline aktif pada port
:8081")
        http.ListenAndServe(":8081", nil)
}
```

Panggil *go startRESTServer(client)* di dalam fungsi main() sebelum atau secara paralel dengan server gRPC, lalu restart layanan.

Langkah 3.2: Penulisan Skenario k6 & Eksekusi Benchmarking

Penanggung Jawab: Mahasiswa D (DevOps Lead).

1. Buat berkas uji beban REST tests/benchmark/k6-rest-test.js:

```
import http from 'k6/http';
import { check } from 'k6';

export const options = {
  stages: [
    { duration: '10s', target: 50 },  // Ramp up 50 pengguna
    { duration: '20s', target: 200 }, // Spike ke 200 pengguna
    { duration: '10s', target: 0 },   // Cool down
  ],
};

export default function () {
  const res =
http.get('http://localhost:8081/api/v1/drugs/check?drug_code=MED-AMX-500
');
  check(res, {
    'status is 200': (r) => r.status === 200,
    'has valid json': (r) => r.json().drug_code === 'MED-AMX-500',
  });
}
```

2. Buat berkas uji beban gRPC tests/benchmark/k6-grpc-test.js:

```
import grpc from 'k6/net/grpc';
import { check } from 'k6';

const client = new grpc.Client();
client.load(['../../proto'], 'pharmacy.proto');
```

```
export const options = {
  stages: [
    { duration: '10s', target: 50 },
    { duration: '20s', target: 200 },
    { duration: '10s', target: 0 },
  ],
};

export default function () {
  client.connect('localhost:50051', { plaintext: true });

  const response =
client.invoke('medcore.pharmacy.v1.PharmacyService/CheckDrugAvailability
', {
    drug_code: 'MED-AMX-500',
    quantity_needed: 10,
  });

  check(response, {
    'status is OK': (r) => r && r.status === grpc.StatusOK,
    'has valid stock': (r) => r && r.message.is_available === true,
  });

  client.close();
}
```

3. Jalankan pengujian secara bergantian menggunakan utilitas k6:

```
# Uji REST Baseline
k6 run tests/benchmark/k6-rest-test.js

# Uji gRPC Protocol Buffers
k6 run tests/benchmark/k6-grpc-test.js
```

Catat metrik berikut dari output terminal k6:

- http_req_duration vs grpc_req_duration (rata-rata, p90, dan p95).
- iterations / throughput (jumlah request per detik / RPS).
- data_received dan data_sent (total konsumsi transfer data jaringan).

Langkah 3.3: Inspeksi Wire-Size Payload

Target: Mengukur perbandingan byte fisik payload serialisasi antara teks JSON vs biner Protocol Buffers.

Jalankan script evaluasi payload berikut via Go di terminal:

```go
// tests/benchmark/inspect_payload.go
package main

import (
	"encoding/json"
	"fmt"
	pb "pharmacy-service/pb"
	"google.golang.org/protobuf/proto"
)

func main() {
	// Objek Representasi yang Sama
	respProto := &pb.CheckDrugResponse{
		DrugCode:    "MED-AMX-500",
		IsAvailable: true,
		CurrentStock: 500,
		UnitPrice:   3500.0,
		Message:     "Stok obat mencukupi",
	}

	protoBytes, _ := proto.Marshal(respProto)

	respJSON := map[string]interface{}{
		"drug_code":    "MED-AMX-500",
		"is_available": true,
		"current_stock": 500,
		"unit_price":   3500.0,
		"message":      "Stok obat mencukupi",
	}
	jsonBytes, _ := json.Marshal(respJSON)

	fmt.Println("================ ANALISIS WIRE-SIZE PAYLOAD ================")
	fmt.Printf("Ukuran Payload JSON (HTTP/1.1)    : %d Bytes\n", len(jsonBytes))
	fmt.Printf("Ukuran Payload Protobuf (HTTP/2)  : %d Bytes\n", len(protoBytes))
	efficiency := (1.0 - float64(len(protoBytes))/float64(len(jsonBytes))) * 100.0
	fmt.Printf("Efisiensi Reduksi Ukuran Kawat    : %.2f%%\n", efficiency)

fmt.Println("============================================================")
```

```
}
```

Observasi Laboratorium: Catat hasil persentase efisiensi pada logbook pengujian.

Langkah 3.4: Studi Kasus Mandiri Sesi III: RPC Deadline Timeout Context

Problem Statement:

Dalam kondisi beban rumah sakit yang padat, basis data farmasi mengalami bottleneck sehingga query memakan waktu 3 detik. Layanan pendaftaran poliklinik tidak boleh membiarkan thread pendaftaran tertahan selamanya (*cascading failure*). Layanan client harus memutus koneksi jika respon melebihi batas toleransi klinis (misal: 500 ms).

Tugas Mandiri Kelompok:

1. Tambahkan time.Sleep(2 * time.Second) pada server gRPC *Pharmacy Service* untuk menyimulasikan bottleneck.
2. Pada *Appointment Service*, set context.WithTimeout(context.Background(), 500*time.Millisecond).
3. Jalankan pemanggilan RPC dan buktikan bahwa client langsung memutus panggilan dengan error status code: codes.DeadlineExceeded.
4. Analisis bagaimana penanganan timeout ini mencegah fenomena kehabisan thread koneksi (*connection starvation*) pada layanan pemanggil.

Langkah 3.5: Git Freeze & Sign-Off Milestone Terpadu

1. Seluruh kode program, DDL, skrip benchmark, dan catatan logbook di-commit ke Git repositori tim:

```
git add .
git commit -m "feat(milestone-1-2): complete domain isolation, grpc
sync, and k6 benchmark"
git push origin main
```

2. Demonstrasikan hasil uji k6 dan penanganan deadline exceeded di depan Asisten/Dosen Pengampu untuk penilaian akhir modul 01 & 02.

6. LATIHAN MANDIRI & ANALISIS NON-TRIVIAL

Setiap mahasiswa secara individu wajib melengkapi laporan mandiri dengan analisis mendalam:

1. **Analisis Teorema CAP pada Konteks Medis:** Mengapa sistem peresepan dan tagihan MedCore lebih condong memilih *Consistency & Partition Tolerance (CP)* daripada *Availability & Partition Tolerance (AP)*? Jelaskan dampaknya jika terjadi partisi jaringan antar kontainer!
2. **Kajian Arsitektur HTTP/2 Multiplexing:** Gambarkan diagram interaksi bagaimana satu koneksi TCP tunggal pada gRPC mampu menangani 100 concurrent request secara paralel tanpa mengalami *Head-of-Line (HoL) Blocking* di lapisan aplikasi, bandingkan

dengan mekanisme *HTTP/1.1 Pipelining*!